

LEMMON PRODUÇÕES

Lemmon Agentes

Manual do Sistema

Versão: v1.18

Data: 2026-05-07

Cliente principal: Hator Clinic

Documento vivo · sempre que uma função for adicionada,
este manual deve ser atualizado e um novo PDF gerado.

LEMMON AGENTES — Manual do Sistema

Versão atual: v1.18 **Última atualização:** 2026-05-07 **Mantido por:** Calebe Alves / Lemmon Produções

Este é o documento de referência viva do sistema Lemmon Agentes. Sempre que uma função nova for implementada ou um épico fechar, este manual deve ser atualizado e uma nova versão de PDF gerada em [docs/releases/](https://docs.releases/).

Histórico de versões

Convenção: versões mais novas no topo. Cada release lista o que mudou em relação à anterior, mantendo histórico completo.

v1.18 — 2026-05-07

BLOCO 6 — T90 V2: barra de progresso + ETA por agente.

- **Backend** GET /sessoes/medianas?agente=X : lê últimas 20 sessões de `historico/dashboard/`, calcula mediana de `duracoes_segundos[agente]`. Cache in-memory TTL 60s. Retorna `{mediana_segundos, amostras}` ou `null` se `< 3` amostras.
- **useChat.ts** — **progress state:** `agentProgress` (0–100) e `agentProgressMeta` (`mediana/elapsed/amostras`) por agente. `setInterval(200ms)` em `agent_start`, `snap 100%` em `agent_done`. Cleanup em `abort()`, `ws.onclose`, e `useEffect unmount` — sem memory leak.
- **FALLBACK_MEDIANAS:** `{otto:20, heitor:40, salles:30, sonia:30, aya:15, pedro_abrahaao:25, renata:30}` em segundos, usado quando endpoint retorna null.
- **Salles A/B:** quando `agentConfig.salles.alternativas` `≡ 3`, mediana $\times 3$ antes de calcular progresso (frontend, ambos os caminhos `fetch` e `catch`).
- **ProgressBar.tsx** : micro barra abaixo de cada bubble. Cor do agente, animação 200ms. Tooltip: "Tempo médio: Xs · decorrido: Ys · n=Z amostras". Cap 95% antes de `agent_done`; `snap 100%` em `agent_done`.
- **Overloaded:** `elapsed > mediana * 1.5` + ainda não concluído → barra âmbar + texto "Mais lento que o normal".
- **Manual lock:** após `agent_done` em modo manual, barra trava em 100% com "🔒 Aguardando aprovação..." até operador decidir.
- **MacroBar.tsx** : visão geral do pipeline no header. Ícones 🕒/▶/✓/✕ por agente + mini barra para agentes ativos.
- **Documentação:** §3.3 e §4.11 atualizados.

FASE 5 — BLOCO 4: doc + versionamento de materiais espelho (T76 → T77 → T85).

- **T76 — Receitas §6 (7 novas):** 6.7 Modo Remix, 6.8 Fast-track, 6.9 Sandbox, 6.10 A/B Salles, 6.11 Briefing Reverso, 6.12 Cortes-prontos, 6.13 Gate Espelho. Cada receita com 4–6 passos, referenciando a feature real no código.
 - **T77 — §10 "Como adicionar cliente espelho":** Guia de 8 etapas do onboarding: `onboard_cliente.py`, dossiê, transcrições, system prompt, instânciação Python, snippet TS em `agents.ts`, alias em `AGENTE_ALIAS`, teste CLI.
 - **T85 — Versionamento de materiais primários (código):**
`EspelhoCliente._carregar_material_primario()` calcula SHA-256 (primeiros 12 chars) do material combinado e registra em `self._material_hash`. Campo `material_hash` adicionado ao resultado de cada execução (`AgenteResultado TypedDict` atualizado). Sessões antigas sem o campo tratam com `?? "?"`. Pasta `inputs/clientes/pedro/historico/` criada como convenção de arquivo de versões antigas.
-

Renata — Social Media (T20): linha editorial Instagram com narrativa conectada.

- **Renata:** Sétimo agente do sistema. Recebe o dossiê da Aya e produz linha editorial Instagram de `n` dias (1 post/dia), com storytelling condicional, datas comemorativas do nicho do cliente e CTA por peça. Apenas 3 formatos: Reels, Carrossel, Stories. Output humano limitado a 5000 chars. Material técnico inclui `linha_narrativa`, `publicacoes`, `descartes`, `estatisticas_mix`.
 - **core/calendario_br.py:** Banco de datas comemorativas BR filtradas por nicho. Inclui `DATAS_FIXAS` (54 datas), `DATAS_MOVEIS` (2026–2027), `CAMPANHAS_MES_INTEIRO` (Outubro Rosa, Novembro Azul, etc.) e `datas_na_janela(inicio, fim, nichos)`.
 - **Modos de operação:** `pipeline` (recebe `dossie_aya` + `roteiro_salles` + `analise_sonia` + `diretrizes_heitor`) e `solo` (contexto livre — retorna 3 perguntas se contexto raso, gera direto se contexto rico).
 - **Descartes:** Material não aproveitado salvo em `outputs/renata/estoque/<ts>_descartes.txt` para reuso posterior.
 - **ConfigSidebar:** Toggle "editorial (~\$0.20)" + range de duração 1–60 dias. Dispara via `config.renata.{incluir, duracao_dias}` no payload WS.
 - **Frontend completo:** sprite SVG (roupa coral #e11d48, prancheta com post-its coloridos), ROLES, IDLE_QUOTES, posições no escritório.
 - **nichos_calendario no EspelhoCliente:** campo opcional `list[str]` para filtrar datas BR relevantes ao nicho do cliente. Configurado como `nacional`, `saude`, `mulher`, `medico` no dossiê de Pedro.
 - **8 smoke tests pytest** (padrão T67) cobrindo: `datas_na_janela`, `pipeline`, modo solo raso, validações de input.
-

FASE 5 — BLOCO 3: tipagem, hooks TS e refatoração de agentes (T74 → T75 → T82 → T81 → T73).

- **T74 — api-client.ts centralizado:** `dashboard/lib/api-client.ts` criado com `apiFetch<T>` utilitário único e 7 funções tipadas exportadas (`fetchBriefingReverso` , `fetchCortesProntos` , `fetchHistorico` , `fetchCalibragemPedro` , `postCalibragemPedro` , `fetchShare` , `postComentario`). As 6 páginas do dashboard refatoradas para usar o `client` (eliminados ~80 `fetch()` inline repetidos).
- **T75 — useApiQuery hook:** `dashboard/lib/use-api-query.ts` exporta `useApiQuery<T>{fn, deps}` genérico retornando `{ data, loading, error, reload }`. Padroniza o ciclo de `loading/error` em todas as páginas que consultam API.
- **T82 — HistoryPanel splitado:** `dashboard/components/history/HistoryPanel.tsx` de 582 linhas dividido em 4 arquivos: `FilterBar.tsx` (filtros de período/origem), `SessionCard.tsx` (card de sessão), `SessionDetail.tsx` (painel de detalhes expandido), `HistoryPanel.tsx` reduzido para 146 linhas como orquestrador.
- **T81 — Sônia migra para _chamar_api da base:** As 3 chamadas diretas a `self.client.messages.create()` em `agentes/sonia.py` migradas para `self._chamar_api()`. Imports de `APIError` , `AuthenticationError` , `RateLimitError` e `Custo` removidos do arquivo. Teste diferencial confirmou output idêntico ao baseline pré-refatoração.
- **T73 — AgenteResultado TypedDict + annotations:** `core/tipos.py` criado com `AgenteResultado(TypedDict, total=False)` contendo 4 campos base garantidos (`output_humano` , `output_tecnico` , `custo_total_usd` , `duracao_segundos`) mais campos extras comuns opcionais. Todos os agentes e `EspelhoCliente` anotam `executar()` → `AgenteResultado`. `AgenteBase` atualiza tipo de retorno abstract. Returns com campos extras usam `cast()`. `_chamar_api_chain` e `_somar_custo` documentados com distinção explícita: `chain` para chamadas independentes (mesma entrada), `somar+_chamar_api` individual para chamadas dependentes (output N-1 alimenta N).

FASE 5 — BLOCO 1: refatoração de dívida técnica (T61 → T62 → T63 → T64 → T65 → T78).

- **T61 — Modularizar api_server.py:** Monólito de ~1317 linhas quebrado em `api/` com 10 módulos: `main.py` (FastAPI + middlewares), `deps.py` (globals compartilhados), `schemas.py` (modelos Pydantic), `storage.py` (persistência de sessões), `ws_helpers.py` (utilidades WebSocket), `ws_chat.py` , `ws_reuniao.py` , `ws_mesa.py` e routers por domínio (`historico` , `exportar` , `exemplares` , `auxiliares` , `transcrever` , `share` , `calibragem`). `api_server.py` vira shim de 1 linha. Output idêntico.
- **T62 — print() → logger:** Substituídos todos os `print()` restantes em `agentes/aya.py` , `agentes/heitor.py` , `agentes/sonia.py` e `core/espelho.py` por `self.logger.info()` / `self.logger.warning()` . Duplicatas (`print` + `logger.warning` no mesmo bloco) eliminadas. Output dos agentes inalterado.
- **T63 — Extrair CSS do exportador:** String inline de ~455 linhas do AURA Design System extraída de `core/exportador_aya.py` para `core/templates/aura.css` . Leitura via

`Path(__file__).parent / "templates" / "aura.css"` em tempo de importação. CSS byte-idêntico ao anterior.

- **T64 — Splitar OfficeScene.tsx:** Componente SVG isométrico de 1627 linhas dividido em 5 arquivos em `dashboard/components/office/`: `constants.ts` (temas, coordenadas, ROLES, IDLE_QUOTES), `SpeechBubble.tsx`, `WorkRoom.tsx` (estúdio + todos os móveis), `MeetingRoom.tsx` e `ReceptionRoom.tsx`. `OfficeScene.tsx` reduzido para 594 linhas com apenas posições de personagens, estado de movimento e componente principal. TypeScript sem erros.
 - **T65 — Splitar ChatPanel.tsx:** Componente de 1493 linhas reorganizado: `MessageBubble.tsx` recebe `exportTxt`, `UserMessage` e `AgentMessage`; `ConfigSidebar.tsx` recebe o sidebar de configurações com interface própria (sem `Props['onUpdateConfig']`). `ChatPanel.tsx` importa dos novos módulos e reduz para 1247 linhas. TypeScript sem erros.
 - **T78 — Heitor usa _chamar_api da base:** As 3 chamadas diretas a `self.client.messages.create()` em `agentes/heitor.py` foram migradas para `self._chamar_api()` da base, que centraliza exception handling (`formatar_erro_anthropic`), timing e cálculo de custo. Imports de `APIError`, `AuthenticationError`, `RateLimitError` e `Custo` removidos do arquivo. Output do agente inalterado.
-

v1.13 — 2026-05-06

FASE 4 — T60: polimento UI (3 fixes).

- **Barra "Uso de agentes" sem rótulo (T60-1):** `Dashboard /saude` usava `agent.color` como cor do texto — Aya tem `#18181b` (quase preto) invisível sobre fundo escuro. Corrigido: label usa `text-stone-300` consistente, com bolinha colorida à esquerda para manter identidade visual do agente. Todos os labels visíveis em todos os temas.
 - **Speech bubble "Dossiê pronto!" sem affordance de clique (T60-2):** Balão de fala era visualmente idêntico ao decorativo mas aparecia em zona clicável do agente. Adicionado `cursor: pointer` explícito ao balão e hover state via CSS (`.speech-bubble-group:hover .speech-bubble-rect` → stroke mais espesso + drop-shadow). O clique já funcionava (`toggleAgent`) — agora é visualmente óbvio.
 - **Outros achados visuais (T60-3):** Sem achados adicionais registrados nesta rodada de testes.
-

v1.12 — 2026-05-06

FASE 4 — T56+T57+T58+T59: QA, TTS, share e telemetria.

- **data-testid no Whiteboard (T56):** `<g data-testid="whiteboard">` adicionado ao container do componente SVG. QA pode usar `document.querySelector('[data-testid=whiteboard]')` ao invés de `[class*=whiteboard]` que falha em elementos `<g>`. Confirmação visual de preenchimento das barras depende de rodar pipeline no Chrome.
- **TTS robusto com detecção de voz pt-BR (T57):** `window.speechSynthesis.speak()` silenciava sem erro quando sem voz pt-BR instalada. Reescrito: enumera vozes via `getVoices()`, detecta pt-BR ou pt por `lang.startsWith`, reporta erro no estado (`ttsError`)

se nenhuma encontrada. Ícone alterna ▶/⏸ (amber quando falando). Console.log das vozes disponíveis facilita diagnóstico em novos ambientes.

- **Página /share/[token] lê err.detail (T58):** Erros da API agora exibem mensagem legível em vez de "Falha na análise" genérica, consistente com T52.
 - **Telemetria de latência por agente (T59):** Cada execução de agente no WebSocket pipeline agora registra duração em `duracoes[name]._salvar_sessao` persiste `"duracoes_segundos": {...}` no JSON da sessão. `HistoryPanel` exibe 🕒 Xs ao lado do custo de cada agente (amber quando >120s). Sessões antigas sem `duracoes_segundos` são compatíveis via `guard ?? {}`.
-

v1.11 — 2026-05-06

FASE 4 — T51+T54+T55: menções, URL e tags.

- **Alias @pedro em reunião (T51):** `_parse_mentions` agora usa `AGENTE_ALIAS` dict para casar nome curto com id composto. `@pedro` aciona Pedro Abrahão; `@pedro_abrahao` (id completo) também funciona. Adicionar aliases novos em `AGENTE_ALIAS` ao onboar novos clientes espelho.
 - **URL /cortes verificada (T54):** Rota existe, build confirma HTTP 200, link do header ✂ abre a página sem 404. O 404 visto no teste E2E era de estado anterior. Verificado em 2026-05-06.
 - **Tags com log e fallback heurístico (T55):** Bloco de tags substitui `except: pass` por `_log.warning` com tipo e mensagem (visível nos logs do uvicorn). Fallback heurístico: quando Haiku falha, extrai 3 palavras mais frequentes do briefing (filtradas por stopwords pt-BR) e envia como chips com prefixo `auto: .Evento tags_sugeridas_falhou` enviado para rastreamento. Feature degrada graciosamente: operador sempre recebe algum sinal.
-

v1.10 — 2026-05-06

FASE 4 — T52+T53: erros Anthropic legíveis (bloco crítico sistêmico).

- **Helper formatar_erro_anthropic (T53):** `core/agente_base.py` centraliza a formatação de erros do SDK Anthropic. `_chamar_api` e `_chamar_api_stream` usam tipos específicos (`APIError`, `APIConnectionError`, `AuthenticationError`, `RateLimitError`) — sem `Exception` genérico que mascararia bugs próprios. Mensagens pt-BR claras para `overloaded`, `rate_limit`, `auth inválida` e `sem conexão`. Chat nunca exibe `{'type': 'error', ...}` como string Python.
 - **Endpoints LLM protegidos (T52):** `/sugerir_pipeline`, `/briefing_reverso` e `/cortes_prontos` agora envolvem a chamada Anthropic com os mesmos tipos específicos → `HTTPException(503, detail=<mensagem legível>)`. Frontend de cada página lê `err.detail` e exibe a mensagem no estado de erro (antes mostrava "Failed to fetch" ou string genérica).
-

FASE 3 — T45 a T50: robustez, UX e polish.

- **Piso no autorizar_custo (T45):** Valor 0 ou negativo no payload `autorizar_custo` agora é clampeado para mínimo \$0.10, evitando loop infinito de "cap atingido".
 - **Risco vermelho Heitor via campo estruturado (T46):** Detecção lê `risco_geral` do `output_tecnico` (dict) antes de fazer fallback no emoji 🚫 — robusto a mudanças de formatação do modelo.
 - **Autocomplete session_id na calibragem (T47):** Página `/calibragem` busca `/historico` ao montar e alimenta `<dataList>` com últimas 20 sessões que incluíram Pedro. Entrada livre continua possível.
 - **Wizard idleQuote com placeholder TODO (T48):** `onboard_cliente.py` gera `idleQuote`: `'TODO: defina a frase de fundo de {nome_curto}'` em vez de frase genérica — força o operador a personalizar antes de colar o snippet.
 - **Agendamento do Pulse semanal (T49):** §5.7 do manual tem instruções completas de `cron` e `launchd` para agendar o pulse toda segunda às 6h no macOS. O script já existia; a automação estava pendente de documentação.
 - **Página `/share/[token]` renderizada no Next.js (T50):** Novo endpoint `GET /share/{token}.json` retorna JSON puro. Página Next.js renderiza com branding Lemmon (header com logo, seções por agente, formulário de comentários). Sem redirecionamento para outro domínio. Endpoint HTML do FastAPI mantido como fallback.
-

FASE 3 — Bloco crítico (T40+T41+T42): QA e segurança do link de aprovação.

- **Corrigir endpoint `/share` (T40):** `POST /share` retornava sempre 404 porque o loop buscava sessões em `HISTORICO_DIR/` (raiz) usando campo `session_id` que não existe nos JSONs. Corrigido para leitura direta em `HISTORICO_DIR/dashboard/{session_id}.json`. Toda a feature de link de aprovação (T36) agora funciona.
 - **Escape XSS no `/share/{token}` (T41):** Briefing, respostas dos agentes, nome do autor e texto dos comentários eram interpolados no HTML sem escape — vulnerabilidade XSS direta. Aplicado `html_escape()` em todos os campos. Adicionada validação no `ComentarioPayload`: `autor` máx 80 chars, `texto` máx 2000 chars. Guard: comentário vazio após `.strip()` retorna 400. Cap: máx 20 comentários por share.
 - **Regularizar versionamento de PDFs (T42):** Decisão Opção A documentada no CHANGELOG — ausência de PDFs v1.3 a v1.6 é aceita; estado consolidado em `MANUAL_v1.7`. A partir desta versão, todo bump de manual deve gerar PDF imediatamente com `python docs/gerar_pdf.py`.
-

Épico G — Camada visual.

- **Whiteboard ao vivo (T31):** O quadro branco da sala de trabalho responde ao pipeline em tempo real. Enquanto um agente processa, barras horizontais na cor daquele agente vão se

preenchendo da esquerda para a direita, proporcional ao volume de texto gerado (0–100%). Quando o pipeline fica idle, o whiteboard retorna às decorações estáticas. Implementado sem texto projetado no plano isométrico — a metáfora visual é de barras de progresso coloridas.

- **Status físico expressivo (T32):** Sprites ganham dois novos estados animados. `done` dispara `animate-celebrate` (pop suave de escala + translação) e exibe badge ✓ verde. `error` dispara `animate-error` (shake lateral) e exibe badge ✗ vermelho. O ponto genérico de status foi preservado apenas para `speaking` (amarelo) e `thinking` (roxo).
- **Mic destaque em reunião (T33):** Quando um agente está em `inMeeting` e com status `speaking`, dois anéis pulsantes (keyframe `mic-ring`) irradiam ao redor do sprite na cor do agente, e um ícone de microfone aparece acima da cabeça. O efeito é exclusivo do modo reunião para não confundir com o pipeline normal.

v1.6 — 2026-05-06

Épico H — Multimodal e aprovação de cliente.

- **Upload de áudio como briefing (T34):** Botão 🎵 no textarea do input aceita arquivos `.mp3`, `.m4a`, `.wav`, `.ogg`. Transcrição automática via Whisper (OpenAI API). Requer `OPENAI_API_KEY` no `.env`. O texto transcrito preenche o input automaticamente. Endpoint: `POST /transcrever` (multipart form). Se a chave não estiver configurada, retorna erro 503 com instruções claras.
- **Narração TTS do dossiê (T35):** Botão "ouvir dossiê" aparece na área de avaliação quando Aya tem resposta pronta. Usa `window.speechSynthesis` nativo do browser em pt-BR — sem API externa, sem custo adicional. Clicar novamente cancela a narração.
- **Link de aprovação de cliente (T36):** Botão "Gerar link de aprovação" na área de avaliação do pipeline. Cria token de sessão e exibe URL copiável. O cliente abre o link e vê uma página HTML limpa (sem custos, sem dados técnicos) com o briefing, as respostas dos agentes e um formulário de comentários. Comentários ficam persistidos e visíveis. Endpoints: `POST /share`, `GET /share/{token}`, `POST /share/{token}/comentar`. Dados em `historico/../../shares/`. Rota dashboard: `/share/[token]`.
- **Calibragem espelho IA x real (T37):** Página `/calibragem` (ícone 🗨️ no header). Registra divergências entre o que o Pedro IA previu e o que o Pedro real disse/reagiu. Cada registro tem: sessão, elemento, predição IA, feedback real, nota de acerto 1–5. Painel exibe KPIs (total, média, % precisão) e histórico completo. Uso: após entregar conteúdo ao Pedro, registrar aqui o feedback real para calibrar o espelho ao longo do tempo. Endpoints: `POST /calibragem_pedro`, `GET /calibragem_pedro`.

v1.5 — 2026-05-06

Épico F — Inteligência operacional e controle de custos.

- **Sugestor de pipeline (T28):** Botão + `sugerir agentes` aparece abaixo do textarea quando o input tem mais de 30 caracteres (modo pipeline). Chama `GET /sugerir_pipeline?briefing=...` — Haiku analisa o briefing e recomenda quais agentes faz sentido convocar. Resultado exibido como chips coloridos com razão curta para cada agente. Botão "Usar

sugestão" sobrescreve a seleção atual e dispensa o card. Endpoint disponível também como API standalone para integrações externas.

- **Routing condicional por risco (T29):** Se Heitor retorna risco  no compliance check, o pipeline envia automaticamente um evento `routing_condicional` com alerta no chat (via bubble Aya). Salles recebe instrução adicional de segurança no prompt ("cuidado redobrado com termos médicos/legais"), reduzindo retrabalho por roteiros que cruzam linhas de compliance.
 - **Custo-cap por sessão (T30):** Controle de orçamento na aba de Configurações do chat. Campo "Limite USD por sessão" — se atingido, pipeline pausa e exibe modal bloqueante com custo acumulado e opções: autorizar +\$0.50, +\$2.00 ou encerrar. Aviso âmbaar não-bloqueante aparece quando custo passa de 80% do cap. Endpoints WS: `custo_aviso` (alerta precoce), `custo_cap_atingido` (pausa); ação cliente: `autorizar_custo` (retoma com novo cap) ou `cancel` (encerra).
-

v1.4 — 2026-05-06

Épico E — Workflows avançados.

- **Modo remix (T22):** Botão  Remix no detalhe de qualquer sessão de pipeline no histórico. Carrega a sessão como retomada, mas pré-seleciona automaticamente Salles+Sônia+Aya (mantendo a tese Otto e o contexto Heitor). Ideal para "mesma estratégia, novo formato/cliente".
 - **Briefing reverso (T23):** Página `/briefing-reverso` (ícone  no header). Cole transcrição, roteiro ou texto já produzido — Otto infere o briefing original, a tese criativa e o posicionamento de marca. Endpoint: `POST /briefing_reverso`.
 - **Comparativo A/B Salles (T24):** Toggle "3 variantes A/B" no ConfigSidebar do Salles. Quando ativo, Salles roda 3 vezes com variações de estilo: padrão, impactante/direto, emocional/pessoal. Os 3 roteiros aparecem em sequência no chat; Sônia vê todos ao ranquear.
 - **Cortes-prontos (T25):** Página `/cortes` (ícone  no header). Cole transcrição longa, selecione durações alvo (15s/30s/60s/90s) — sistema gera tabela de cortes com timestamps, hook e CTA por duração. Endpoint: `POST /cortes_prontos`.
 - **Fast-track / Modo emergência (T26):** Botão  no header do pipeline. Quando ativo: Otto roda em modo resumido, Heitor é pulado (com aviso de risco assumido no chat), gate espelho ignorado. Resultado em <3 min. Atenção: sem Heitor, valide compliance manualmente antes de publicar.
 - **Lab/Sandbox (T27):** Botão  no header do pipeline. Quando ativo: sessão não é salva no histórico, sem sugestão de tags. Ideal para testar ideias sem poluir registros ou virar referência futura.
-

v1.3 — 2026-05-06

Épico C — Memória institucional e saúde do sistema.

- **Pulse semanal (T11):** Script `scripts/pulse_semanal.py` gera relatório semanal em markdown — sessões, custos, tendências e análise narrativa por Aya. Rodável via cron ou

manualmente: `python scripts/pulse_semanal.py --semana 2026-W18`. Output em `outputs/pulse/`.

- **Few-shot curado (T12):** Sessões 5★ podem ter trechos marcados como exemplares. Botão ☆ exemplar aparece em cada resposta de agente nas sessões 5-estrelas do histórico. Exemplares são salvos em `core/exemplares/<agente>.json` e injetados automaticamente no `system_prompt` de cada agente. Endpoints: `POST /exemplares`, `GET /exemplares/{agente}`, `DELETE /exemplares/{agente}/{id}`.
- **Busca semântica (T13):** Antes de enviar um briefing, botão 🔍 ver referências similares (aparece quando input > 20 chars, modo pipeline) busca sessões passadas com briefings similares por TF-IDF de tokens. Endpoint `GET /historico/similar?briefing=...&n=3`.
- **Hall of Fame (T14):** Página `/hall-of-fame` lista todas as sessões 5★ em grid de cards com briefing, agentes, custo e filtros por período e agente. Acessível pelo ícone 🏆 no header.
- **Tags semi-automáticas (T15):** Ao fim de todo pipeline, Haiku sugere automaticamente 3-5 tags descritivas. Chips aparecem na sessão; o operador pode remover tags indesejadas (× em cada chip). Tags aceitas são salvas com a avaliação via `POST /tags`. Endpoint `POST /tags` também disponível para salvar tags sem nota.
- **Dashboard de saúde (T16):** Página `/saude` com KPIs do sistema: sessões totais, custo total e médio, taxa de avaliação, taxa 5★; bar charts CSS de sessões e custo por mês (últimos 6); horizontal bars de uso por agente. Acessível pelo ícone de atividade no header.
- **Histórico filtrável (T17):** FilterBar no painel de histórico com filtros por período (7/30/90 dias), origem (dashboard/reunião), agente envolvido, e nota mínima (inclui opção "sem avaliação"). Contador no cabeçalho mostra `filtradas/total` quando filtro ativo. Botão "limpar (N)" reseta tudo.

v1.2 — 2026-05-06

Épico B — Pedro como gate de qualidade.

- **Gate espelho (T9):** Após o Salles e antes da Sônia, Pedro pode validar automaticamente se o roteiro está fiel à voz/posicionamento do cliente. Configurável por sessão no painel do Salles: `off` (padrão, sem gate), `auto` (bloqueia se veredicto 🛑), `manual` (sempre pede aprovação). O veredicto aparece no chat com badge 🟢/🟡/🛑. Em modo `auto`, roteiros com 🛑 bloqueiam o pipeline e o operador decide se continua.
- **Mesa redonda stress test (T10):** Botão 🗪 mesa no toolbar de reunião. Escreva a tese no campo de input e clique para que cada agente presente questione a tese no seu ângulo específico (estratégia, compliance, roteiro, performance, cliente espelho). Aya sintetiza uma ata executiva. Endpoint: `GET /ws/mesa_redonda`.

v1.1 — 2026-05-06

Épico A — Família de espelhos de cliente.

- **EspelhoCliente (T6):** classe genérica extraída de `PedroAbrahao`. Qualquer cliente espelho é agora uma instância paramétrica de `EspelhoCliente(id, nome, material_dir, ...)`. Pedro virou uma factory function em `agentes/pedro_abrahao.py`. Material primário migrado para `inputs/clientes/pedro/`.

- **Wizard de onboarding (T7):** `onboard_cliente.py` — CLI interativo (ou com args `--id --nome --nicho --cor`) que cria automaticamente dossiê, transcrições, system prompt e pasta de outputs para um novo cliente espelho. Também gera snippet TypeScript para `agents.ts` e exemplo de instanciação Python.
- **Salas temáticas (T8):** sala de reunião muda paleta visual (tapete, brilhos) conforme o cliente ativo. Seletor de cliente no header da cena — exibe nome + cor do cliente; botão "trocar 🔄" aparece quando há mais de um cliente registrado. Para adicionar tema de novo cliente: incluir entrada em `MEETING_THEMES` em `OfficeScene.tsx` .

Para adicionar um novo cliente espelho:

```
python onboard_cliente.py # wizard interativo
# ou com args:
python onboard_cliente.py --id marina --nome "Marina Costa" --nicho "nutricionista"
```

Depois preencher `inputs/clientes/<id>/dossie.md` , colar transcrições reais, e adicionar o snippet TypeScript gerado em `dashboard/lib/agents.ts` e a entrada de tema em `dashboard/components/office/OfficeScene.tsx > MEETING_THEMES` .

v1.0 — 2026-05-05

Primeira versão publicada. Documenta o estado atual do sistema:

- 6 agentes operacionais: Otto, Heitor, Salles, Sônia, Aya, Pedro
- Pipeline sequencial e Modo Reunião conversacional
- Modo Manual (aprovação step-by-step)
- Retomada de sessão (continuar trabalho anterior)
- Histórico persistido + avaliação por estrelas
- Upload de imagem com descrição automática (visão Haiku)
- Speech-to-text no input
- Streaming nativo da Anthropic em modo Reunião
- URL backend centralizada (`API_URL` / `WS_URL`)
- 6 CLIs individuais + pipeline completo via terminal

Correções incorporadas (T1–T5 do PLANO_ACAO): custo do Otto padronizado, Aya recebe contexto técnico, `/avaliar` lê do disco, URL centralizada, streaming nativo em Reunião.

Sumário

1. Visão geral
2. A equipe — 6 agentes
3. Como o sistema funciona — modos de operação
4. Funcionalidades do dashboard
5. CLI direto (terminal)

- 6. Receitas — workflows recomendados
 - 7. Roadmap — o que está vindo
 - 8. Apêndice — custos, limites, configuração
 - 9. Como atualizar este manual
-

1. Visão geral

O **Lemmon Agentes** é o sistema interno de produção de conteúdo da Lemmon Produções. Em vez de Calebe escrever roteiros do zero a cada projeto, ele convoca uma equipe de agentes especializados que processam o briefing em camadas: estratégia, compliance, roteiro, performance, compilação, e validação por espelho de cliente.

O sistema funciona em dois modos principais. O **Pipeline** corre em sequência fixa e é otimizado para entrega rápida de uma sessão completa. O **Modo Reunião** é conversacional, multi-turno, e permite discussão livre entre os agentes com menções estilo Slack. Os dois usam a mesma equipe de fundo, mas com posturas diferentes.

Tecnicamente o sistema é composto por: backend Python (FastAPI + WebSocket) que orquestra os agentes via API da Anthropic; dashboard Next.js/React com escritório virtual onde os personagens vivem; e histórico persistido em JSON local para auditoria, avaliação e retomada de sessões. Tudo roda no computador do operador — sem cloud, sem servidor.

Filosofia: o sistema não substitui o operador. Ele acelera o pensamento ao dar uma equipe de cabeças especializadas trabalhando em paralelo. Calebe continua sendo o diretor — os agentes são os redatores assistentes.

2. A equipe — 6 agentes

A equipe atual tem 6 personagens. Cinco entram no pipeline padrão (Otto → Heitor → Salles → Sônia → Aya); o sexto, Pedro, é convocado sob demanda em modo Reunião.

2.1 Otto — Estrategista

Cor: azul-marinho · **Classe RPG:** Analista

Papel. Otto é o primeiro a tocar o briefing. Ele decodifica o que o cliente pediu, o que o cliente NÃO disse, identifica o conflito central, a insegurança subjacente, e produz uma tese criativa. Saída técnica em formato estruturado (tool use forçado), mais uma versão humana em markdown.

Quando usar. Sempre que houver briefing novo. Otto é o ponto de partida do pipeline. Em modo Reunião ele entra quando você quer um olhar estratégico sobre uma ideia ou validação de tese.

Configurações.

Parâmetro	Valores	Descrição
<code>modo_visual</code>	<code>completo</code> , <code>resumido</code> , <code>mínimo</code>	Tamanho do output. Completo dá tudo; resumido só tese + conceito; mínimo é tese só

Exemplo de input. "Cliente é uma clínica de medicina personalizada para mulheres 35-55. Quer um vídeo curto sobre tratamento de menopausa que não soe como propaganda."

Exemplo de output (resumido).

TESE: a verdadeira reposição é de identidade, não de hormônio.

CONCEITO: "A mulher que você vai voltar a ser" — formato testemunhal, posicionamento da clínica como restauradora de algo perdido, não vendedora de tratamento.

Custo médio. \$0.02–0.08 por execução (briefing simples a complexo).

2.2 Heitor — Compliance

Cor: verde-musgo · **Classe RPG:** Guardião

Papel. Heitor é a barreira contra falar bobagem. Verifica termos críticos contra Anvisa, conselhos profissionais (CFM, CRM, CFO, CRO, CFN, CFF, CFP, COFFITO), e diretrizes da Meta. Pode buscar em domínios oficiais para checar se uma reivindicação se sustenta.

Quando usar. Para qualquer conteúdo que vá ao público em saúde, beleza, suplementação, terapias. Em pipeline interno (proposta, pitch da Lemmon) Heitor pode ser pulado.

Configurações.

Parâmetro	Valores	Descrição
<code>max_buscas</code>	1–10	Quantas buscas web Heitor pode fazer (padrão 3, profundo 6)
<code>secundarias</code>	bool	Permite buscar em fontes não-oficiais (jornalismo, marketing)

Saída. Risco geral verde / amarelo / vermelho. Lista de termos críticos identificados. Sugestões de reescrita. URLs das fontes consultadas.

Comportamento de confirmação. Antes de buscas longas pede confirmação ao operador via WebSocket (callback `_make_confirmacao_callback`). Em modo manual o operador aprova cada busca; em modo automático com aviso de custo acima do threshold (\$0.50), pede confirmação.

Custo médio. \$0.20–0.40 por execução com buscas. Sem buscas, \$0.05–0.10.

2.3 Salles — Roteirista

Cor: terracota · **Classe RPG:** Criativo

Papel. Transforma a tese do Otto em roteiro filmável. Trabalha com formatos pré-definidos. Sabe questionar a estratégia (ver `core/discussao.py` — Salles pode rodar uma rodada de discussão com o Otto antes de escrever, contestando viabilidade técnica e foco narrativo).

Quando usar. Sempre que a saída final for um roteiro a ser produzido. Em validação de pitch ou apresentação institucional, Salles é dispensável.

Configurações.

Parâmetro	Valores
<code>formato</code>	<code>auto</code> , <code>reels</code> , <code>documental</code> , <code>mini-doc</code> , <code>tese</code> , <code>aftermovie</code>

Saída. Roteiro em markdown com bloco-a-bloco, indicação de ação, fala, b-roll sugerido. Estrutura técnica em JSON paralelo (`formato_aplicado`, `num_blocos`, `titulo_roteiro`).

Custo médio. \$0.10–0.25 por roteiro.

2.4 Sônia — Performance

Cor: violeta · **Classe RPG:** Growth

Papel. Avalia o roteiro do ponto de vista de performance: hooks, retenção, CTA, adequação a tendências. Pode buscar tendências atuais em domínios pré-aprovados (Meta business, criadores oficiais). Sugere cortes autônomos a partir do roteiro longo.

Quando usar. Após Salles em qualquer projeto que vá para distribuição em rede social. Em peças institucionais offline, dispensável.

Configurações.

Parâmetro	Valores	Descrição
<code>com_busca</code>	bool	Ativa web search (custo extra)
<code>usar_tendencias</code>	bool	Injeta <code>inputs/tendencias_atuais.md</code> no prompt
<code>modo</code>	<code>cadeia</code> , <code>solo</code> , <code>cortes_apenas</code>	Cadeia = passa por análise master; solo = direto; cortes_apenas = só gera cortes

Saída. Nota master 0–10. Análise por critério. Lista de cortes autônomos com timestamps sugeridos. Peça destaque indicada.

Custo médio. \$0.15–0.25 sem busca; \$0.30–0.50 com busca.

2.5 Aya — Compiladora (Oráculo)

Cor: preto · **Classe RPG:** Oráculo

Papel. Compila os outputs dos outros agentes em um dossiê único, organizado, em markdown.

Não interpreta, não sintetiza, não opina — só organiza. Arquitetura em duas etapas: chamada API que produz cards de resumo + montagem Python pura do markdown final.

Quando usar. Como última etapa do pipeline para entregar um deliverable consolidado. Também em modo Reunião quando você quer perguntar coisas sobre o sistema ou a equipe (em reunião usa `system_prompt_reuniao` reduzido, sem material de compilação).

Configurações. Aya não tem configurações no dashboard hoje. Pode receber `outputs_diretos` (no pipeline já é passado automaticamente desde T2) ou `arquivos_especificos` (CLI).

Saída. Dossiê markdown com cabeçalho, índice, página de resumo dos agentes, e seções completas de cada agente que rodou. **Regra de ouro: agentes ausentes não aparecem no dossiê.**

Custo médio. \$0.05–0.20.

2.6 Pedro — Consultor (Cliente Hator)

Cor: verde-azulado (`#0f766e`) · **Classe RPG:** Cliente

Papel. Espelho IA do Dr. Pedro Abrahão (Hator Clinic). Treinado com dossiê de posicionamento + transcrições reais. Avalia conteúdo da ótica do cliente: voz fiel? posicionamento correto? cacoetes verbais respeitados? Recusa zonas (diagnóstico médico, decisões grandes do negócio, temas íntimos).

Quando usar. Sob demanda em modo Reunião — ele tem flag `reuniaoOnly: true` e não entra no pipeline padrão. Útil para validar roteiro pronto antes de gravação, simular reação do cliente a uma proposta, ou perguntar "como Pedro responderia a X?".

Configurações. No CLI (`pedro_cli.py`):

Parâmetro	Valores
<code>--modo</code>	<code>validacao</code> , <code>consulta</code> , <code>resposta_hipotetica</code>
<code>--contexto</code>	Caminho de arquivo com texto a avaliar (ex: roteiro pronto)

Material primário. Carregado via `EspelhoCliente._carregar_material_primario()` a partir de `inputs/clientes/pedro/dossie.md` + `inputs/clientes/pedro/transcricoes.md`. Injetado em `system_prompt_reuniao` — funciona corretamente em modo Reunião e CLI. A cada execução, um hash SHA-256 (12 chars) do material é calculado e registrado no JSON de histórico como `material_hash` — rastreabilidade de qual versão do dossiê foi usada.

Como instanciar (Python):

```
from agentes.pedro_abrahaao import PedroAbrahaao
pedro = PedroAbrahaao()
resultado = pedro.executar(pergunta="...", modo="validacao")
```

Custo médio. \$0.05–0.20.

2.7 Renata — Social Media

Cor: rosa-coral (`#e11d48`) · **Classe RPG:** Comunicadora

Papel. Recebe o dossiê compilado pela Aya e produz **linha editorial Instagram** com narrativa conectada — em 1–2 páginas que o cliente entende em 2 minutos. Não substitui nenhum outro agente: respeita o roteiro do Salles, o risco do Heitor, a performance da Sônia e a tese do Otto. Costura, organiza e distribui em calendário.

Escopo. Apenas Instagram: Reels, Carrossel, Stories. Cadência: 1 post por dia. Volume: igual à duração da campanha em dias.

Storytelling condicional. Se `tem_arco=true` (campanhas de vendas ou lançamento): usa cliffhangers, callbacks e escalada emocional entre peças. Se `tem_arco=false` (campanhas avulsas ou institucionais): sem forçar narrativa.

Calendário BR. Identifica feriados e datas comemorativas relevantes ao nicho do cliente na janela da campanha via `core/calendario_br.py`. Nichos do cliente Pedro: `nacional`, `saude`, `mulher`, `medico`.

Descartes. Material que não coube no calendário é listado com justificativa e salvo em `outputs/renata/estoque/<timestamp>_descartes.txt`.

Modos de operação:

Modo	Quando usar	Input
<code>pipeline</code>	Após Aya no pipeline completo	<code>dossie_aya</code> ou <code>roteiro_salles</code>
<code>solo</code>	Em Reunião isolada	<code>contexto_solo</code> (pode ser raso)

Modo solo com contexto raso: Renata retorna 3 perguntas DE UMA VEZ antes de gerar.

Quando usar. Pipeline: toggle "editorial" no ConfigSidebar + duração (1–60 dias). Reunião: @renata .

Configurações CLI (renata_cli.py):

Parâmetro	Valores
--modo	pipeline (default), solo
--duracao	1–60 (default: 14)
--inicio	YYYY-MM-DD (default: hoje + 7 dias)
--dossie	Caminho do .md da Aya
--roteiro	Caminho do roteiro do Salles
--cliente	ID do cliente (ex: pedro_abrahao)

Como instanciar (Python):

```
from agentes.renata import Renata
renata = Renata()
resultado = renata.executar(
    modo="pipeline",
    duracao_dias=14,
    dossie_aya=texto_dossie,
    cliente_id="pedro_abrahao",
)
```

Custo médio. \$0.10–0.25.

3. Como o sistema funciona — modos de operação

3.1 Modo Pipeline

Sequência fixa: Otto → Heitor → Salles → Sônia → Aya. Sem volta, sem ramificação. Cada agente recebe o output do anterior, produz o seu, passa adiante. Aya é sempre a última e compila tudo.

Quando usar. Briefing novo, deliverable claro, deadline. Quando você sabe o que quer e só precisa que o sistema entregue.

Como ativar. No dashboard, clique nos agentes que devem entrar no pipeline (cards no header). Modo "pipeline" no toggle do chat panel. Envie o briefing.

Comportamento. WebSocket `/ws/chat` recebe `agents`, `message`, `manual_mode`, `config`. Para cada agente em ordem, executa, faz streaming dos tokens, salva resposta. Ao final, salva sessão completa em `historico/dashboard/<timestamp>_sessao.json` e envia `pipeline_done` com `session_id`.

3.2 Modo Reunião

Conversacional, multi-turno. Você manda mensagem, agentes respondem (em ordem). Próxima mensagem entra como turno seguinte. Histórico persistido client-side (sobrevive reconexão WS) e server-side (salvo a cada turno em `historico/dashboard/<timestamp>_reuniao.json`).

Quando usar. Brainstorm, validação de ideia, debate de direção criativa, discussão entre agentes. Quando você não sabe ainda o que quer, ou quer ouvir várias cabeças antes de decidir.

Como ativar. Toggle "conv." no chat panel. Convoque os agentes que devem participar.

Menções (@). Em modo manual da reunião, só agentes mencionados respondem. Em modo auto, todos respondem em ordem (a menos que um seja mencionado especificamente). Use `@otto`, `@heitor`, etc.

Streaming. Modo Reunião usa streaming nativo da Anthropic (token a token, real). Pipeline ainda usa stream simulado por questão de tool use forçado nos agentes.

3.3 Modo Manual (aprovação step-by-step)

Toggle no header do chat panel. Em vez de o pipeline correr direto até o fim, o sistema pausa após cada agente e espera o operador clicar **Continuar**, **Pular**, **Tentar de novo** ou **Cancelar**.

Quando usar. Em projetos sensíveis ou caros onde você quer ver o output de cada etapa antes de seguir. Também útil para depurar comportamento estranho de um agente.

Aceite por step. Em caso de erro do agente em modo manual, três opções aparecem na barra de aprovação: **Retry** (tenta de novo), **Pular** (segue sem o output desse agente) ou **Cancelar** (encerra pipeline).

Barra de progresso em modo manual. A barra abaixo do bubble do agente sobe normalmente até 95% durante o processamento. Quando o agente termina (`agent_done`), trava em 100% e exibe "🔒 Aguardando aprovação..." enquanto o operador decide. A barra desaparece ao aprovar, pular ou cancelar.

3.4 Retomada de sessão

Você pode pegar uma sessão antiga do histórico, clicar **Retomar**, e o sistema recarrega contexto técnico (análise Otto, diretrizes Heitor, roteiro Salles, etc.) e abre o chat para você continuar.

Quando usar. Cliente voltou com ajuste 3 dias depois. Você quer pegar a tese do Otto e rodar Salles num formato diferente. Você quer só rodar Aya numa sessão antiga que não tinha sido compilada.

Como. Histórico → escolher sessão de pipeline (não funciona com reunião por enquanto) → botão **Retomar**. Banner laranja aparece no chat: "Sessão retomada — selecione agentes e continue". Selecione apenas os agentes que precisam re-rodar e mande o ajuste como nova mensagem.

Detalhe. Ao retomar, se você mandar nova mensagem, ela vira `[INSTRUÇÃO ADICIONAL]` concatenada ao briefing original. Se mandar mensagem vazia (placeholder), o sistema usa só o briefing original.

3.5 Avaliação

Após cada sessão concluída, aparece barra "Como foi essa sessão?" com 5 estrelas. Avaliação salva no JSON da sessão. Pode também adicionar observações e tags.

Quando usar. Sempre que possível — vira combustível para o sistema aprender (Épico C do plano: few-shot curado de 5★).

Onde fica. No JSON da sessão como `avaliacao`, `observacoes_operador`, `tags`. Endpoint `/avaliar` lê do disco (sobrevive a reinício do servidor).

4. Funcionalidades do dashboard

4.1 Upload de imagem

Clipe no canto inferior esquerdo do input. Imagem (JPG/PNG/WEBP, máx 5MB) é descrita automaticamente por Claude Haiku 4.5 e a descrição é injetada no briefing antes de chegar nos agentes. Útil para:

- Foto da clínica → contexto visual entra na análise do Otto
- Print de tendência do Instagram → Sônia pode reagir
- Mood board → Salles pega o tom

Falha silenciosa atualmente. Se a descrição da imagem falha (rate limit, etc.), o briefing segue sem o contexto visual e o operador não é avisado. Polimento C do plano corrige isso.

4.2 Speech-to-text

Microfone no canto direito do input. Usa Web Speech API (`SpeechRecognition`) em pt-BR. Continuous + interim results. Útil para gravar briefing em voz quando você não quer digitar.

Limitação. Funciona melhor no Chrome/Edge. Safari pode falhar. Firefox não tem.

4.3 Exportar sessão

Botão de download no header do chat panel. Gera arquivo `lemmon_sessao_<data>.txt` com todas as mensagens da sessão atual, incluindo custos por agente. Útil para arquivar fora do sistema, mandar pra cliente, anexar em email.

4.4 Histórico

Painel flutuante, ícone de relógio no header geral. Lista as 200 sessões mais recentes (limite atual). Cada item mostra: timestamp, briefing truncado, agentes usados, custo total, avaliação, e badge se é pipeline ou reunião.

Detalhe da sessão. Click → painel abre detalhe completo, incluindo respostas por agente, custos individuais, e — para reuniões — o histórico cronológico dos turnos.

Filtros (T17). FilterBar acima da lista: período (7/30/90 dias), origem (dashboard/reunião), agente envolvido, e nota mínima. Contador `filtradas/total` no cabeçalho quando filtro ativo.

4.5 Hall of Fame

Página `/hall-of-fame` (ícone 🏆 no header). Grid de cards com todas as sessões 5★. Filtros por período e por agente. Útil para mostrar ao cliente o tipo de trabalho que o sistema produz, ou para inspiração antes de uma nova sessão.

4.6 Dashboard de saúde

Página `/saude` (ícone de atividade no header). KPIs: sessões totais, custo total e médio, taxa de avaliação e taxa 5★. Bar charts de sessões e custo por mês (últimos 6). Horizontal bars de uso por agente com percentual.

4.7 Referências similares (busca semântica)

No modo pipeline, quando o campo de input tem mais de 20 caracteres, aparece o botão "🔍 ver referências similares". Clicando, o sistema busca sessões passadas com briefings semanticamente próximos (TF-IDF de tokens pt-BR). Retorna até 3 resultados com briefing truncado, avaliação e score de similaridade.

4.8 Tags sugeridas

Ao fim de cada pipeline, Haiku gera automaticamente 3-5 tags descritivas. Aparecem como chips logo acima do bloco de avaliação. Clique x em qualquer chip para dispensar a tag — a lista restante é salva imediatamente. As tags aceitas também são incluídas quando você avalia com estrelas.

4.9 Painéis flutuantes

Tanto o ChatPanel quanto o HistoryPanel são arrastáveis (drag pelo header) e redimensionáveis (handles nas bordas). Posições persistem na sessão atual; resetam ao recarregar.

4.11 Barra de progresso + ETA (T90)

Durante a execução do pipeline, cada agente exibe uma micro barra de progresso abaixo do seu bubble de resposta. A barra sobe de 0% a 95% com base na mediana histórica de duração daquele agente (calculada nas últimas 20 sessões). Ao chegar em 100%, ela indica conclusão e desaparece.

MacroBar (visão geral). Acima das mensagens do pipeline, uma barra de status mostra todos os agentes em execução com ícone de estado: ⌚ aguardando, ▶ processando, ✓ concluído, ✕ erro. Agentes ativos exibem uma mini barra de progresso individual.

ETA e detalhes. Passe o cursor sobre qualquer barra para ver o tooltip: `Tempo médio: 30s` · `decorrido: 18s` · `n=15 amostras`.

Overloaded (âmbar). Se o agente ultrapassar $1,5\times$ a mediana histórica sem terminar, a barra muda para âmbar e exibe "Mais lento que o normal". Não implica erro — apenas sinaliza que essa execução está mais devagar que o habitual.

Fallback sem dados. Enquanto o sistema não acumula 3 amostras para um agente, usa tempos fixos de referência: Otto 20s, Heitor 40s, Salles 30s, Sônia 30s, Aya 15s, Pedro 25s, Renata 30s.

Salles A/B. Quando `config.salles.alternativas = 3`, a mediana estimada é multiplicada por 3 automaticamente — o agente produz 3 roteiros completos, demora proporcionalmente.

Endpoint backend. `GET /sessoes/medianas?agente=X` retorna `{mediana_segundos, amostras}` ou `null` se < 3 amostras. Cache in-memory TTL 60s.

4.10 Escritório virtual

Cena RPG com sprites dos agentes em mesas. Quando você convoca um agente, ele caminha da mesa para a sala de reunião. Status físico (idle, thinking, speaking, done, error) reflete em cor e animação. Idle quotes aparecem em bolas de fala periodicamente.

Roadmap: os Épicos G do plano aprofundam a camada visual — whiteboards que se preenchem em tempo real, mic destacado em quem fala, salas customizadas por cliente.

5. CLI direto (terminal)

Cada agente tem um CLI próprio para uso fora do dashboard. Útil para automação, scripts, debugging.

5.1 Otto

```
python otto_cli.py inputs/briefing.txt
python otto_cli.py inputs/briefing.txt --modo-visual resumido
```

5.2 Heitor

```
python heitor_cli.py inputs/conteudo.md --max-buscas 5
python heitor_cli.py inputs/conteudo.md --no-confirm
```

5.3 Salles

```
python salles_cli.py inputs/briefing.txt --formato reels
python salles_cli.py inputs/briefing.txt --formato mini-doc --tags hator,menopausa
```

5.4 Sônia

```
python sonia_cli.py outputs/salles/<roteiro>.md
python sonia_cli.py outputs/salles/<roteiro>.md --com-busca --modo cadeia
```

5.5 Aya

```
python aya_cli.py
python aya_cli.py --nome-projeto "Hator menopausa"
```

5.6 Pedro

```
python pedro_cli.py "como você responderia se uma paciente perguntasse X"
python pedro_cli.py inputs/pergunta.txt --modo validacao --contexto outputs/salles/roteiro.md
```

5.7 Pulse semanal

```
python scripts/pulse_semanal.py                # semana atual
python scripts/pulse_semanal.py --semana 2026-W18 # semana específica
python scripts/pulse_semanal.py --dias 14        # últimos 14 dias
python scripts/pulse_semanal.py --dry-run        # só mostra contexto, sem chamar API
```

Agendamento automático

macOS — cron (toda segunda às 6h):

```
crontab -e
```

Adicionar a linha:

```
0 6 * * 1 cd /Users/calebe/Documents/lemmon-agentes && .venv/bin/python scripts/pulse_semanal.py >> /
```

macOS — launchd (mais confiável, funciona mesmo sem login):

Criar `/Library/LaunchDaemons/com.lemmon.pulse.plist` (ou em `~/Library/LaunchAgents/` para agente de usuário):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>      <string>com.lemmon.pulse</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/calebe/Documents/lemmon-agentes/.venv/bin/python</string>
    <string>/Users/calebe/Documents/lemmon-agentes/scripts/pulse_semanal.py</string>
  </array>
  <key>StartCalendarInterval</key>
  <dict>
    <key>Weekday</key> <integer>1</integer>
    <key>Hour</key>    <integer>6</integer>
    <key>Minute</key>  <integer>0</integer>
  </dict>
  <key>StandardOutPath</key> <string>/tmp/pulse_stdout.log</string>
  <key>StandardErrorPath</key> <string>/tmp/pulse_stderr.log</string>
</dict>
</plist>
```

Carregar:

```
launchctl load ~/Library/LaunchAgents/com.lemmon.pulse.plist
```

5.8 Pipeline completo

```
python pipeline_completo.py inputs/briefing.txt
python pipeline_completo.py inputs/briefing.txt --formato reels --com-aya
python pipeline_completo.py inputs/briefing.txt --profundo --busca-sonia --com-aya
```

Flags úteis: `--sem-heitor`, `--sem-sonia`, `--no-confirm`, `--profundo`, `--sonia-profundo`, `--nome-projeto "X"`.

5.9 Exemplos (few-shot curado)

Gerenciado pelo endpoint `/exemplares`, mas também pode ser inspecionado direto em `core/exemplares/<agente>.json`. Para remover um exemplar problemático: `DELETE /exemplares/{agente}/{id}` via curl ou via código. O limite é 10 exemplares por agente; os 3 mais recentes são injetados no `system_prompt`.

6. Receitas — workflows recomendados

6.1 Roteiro novo do zero (cliente Hator)

1. Abrir dashboard
2. Convocar Otto, Heitor, Salles, Sônia, Aya
3. Modo pipeline, modo manual ativo (recomendado)
4. Enviar briefing
5. Aprovar Otto após ler tese
6. Aprovar Heitor (se risco vermelho, decidir continuar ou parar)
7. Aprovar Salles após ler roteiro
8. Aprovar Sônia
9. Aya compila automaticamente
10. Avaliar sessão (5★ se ficou bom)

Tempo médio: 5–10 min com modo manual, 2–4 min em automático.

6.2 Validação de roteiro pronto

Você já tem um roteiro pronto e quer validação antes de gravar.

1. Modo Reunião
2. Convocar Pedro + Heitor (gate de cliente + gate de compliance)
3. Mensagem: @pedro avalie esse roteiro: [colar roteiro]
4. Pedro responde com observações de fidelidade
5. Mensagem: @heitor passa por compliance: [colar trecho de risco]
6. Decidir ajustes baseado nos dois pareceres

6.3 Brainstorm de tese

Você não tem briefing fechado ainda. Quer pensar em voz alta com a equipe.

1. Modo Reunião
2. Convocar Otto + Salles + Sônia
3. Modo manual da reunião (só responde se @mencionado)
4. Conversar livremente, mencionar quem deve opinar quando

6.4 Variação de algo que já funcionou

1. Histórico → encontrar sessão antiga 5★
2. Botão **Retomar**
3. Modo pipeline com Salles + Sônia + Aya (Otto não precisa rodar de novo)
4. Mensagem: mesma tese, formato Reels em vez de mini-doc
5. Pipeline herda análise estratégica antiga e regenera só o que mudou

6.5 Checagem só de compliance

Você tem um post pronto e só quer ver se passa pelo Heitor.

1. CLI: `python heitor_cli.py inputs/post.md`
2. Ler relatório de risco e termos críticos
3. Ajustar texto e rodar de novo se necessário

6.6 Rodar em background (sem dashboard)

Para automação ou processamento em lote, use o `pipeline_completo.py`:

```
for briefing in inputs/lote/*.txt; do
    python pipeline_completo.py "$briefing" --com-aya --no-confirm
done
```

6.7 Variar uma estratégia que funcionou (Modo Remix)

1. Histórico → encontrar sessão 5★ com tese que funcionou
2. Botão **Retomar** — dashboard envia `resume_context` com análise Otto herdada
3. Convocar apenas Salles + Sônia + Aya (Otto não precisa rodar de novo)
4. Mensagem: novo formato: Reels 30s em vez de mini-doc, mesma tese
5. Salles regenera roteiro a partir da análise Otto herdada
6. Aya compila com referência cruzada à sessão original

6.8 Roteiro emergencial sob deadline (Fast-track)

1. Dashboard → toggle **Fast-track** (ícone ⚡ no header do chat)
2. Convocar Otto + Salles + Sônia + Aya
3. Enviar briefing normalmente
4. Fast-track força Otto em modo resumido e pula **Heitor** automaticamente
5. Pipeline completo em ~50% do tempo normal
6. Aviso aparece no chat: "Heitor pulado — valide compliance manualmente antes de publicar"

6.9 Testar ideia maluca sem poluir histórico (Sandbox)

1. Dashboard → toggle **Sandbox** (ícone  no header do chat)
2. Montar pipeline normalmente, enviar briefing experimental
3. Pipeline roda completo — sessão **não aparece** em `/historico` nem conta para métricas
4. Explorar abordagens arriscadas, linguagem nova, formatos incomuns
5. Para salvar de verdade: desativar sandbox e rodar a versão aprovada

6.10 Comparar 3 abordagens de roteiro (A/B Salles)

1. Convocar Otto + Salles + Sônia + Aya
2. No painel de config do Salles, definir `alternativas: 3`
3. Pipeline roda normalmente até Salles — Salles produz 3 versões (resumo, mini-doc, reel)
4. Chat exibe três bolhas: **salles_v1**, **salles_v2**, **salles_v3**
5. Escolher a versão preferida antes de passar para Sônia
6. Sônia otimiza a versão selecionada; Aya compila só essa

6.11 Reverse-engineer um vídeo concorrente (Briefing Reverso)

1. Abrir página `/briefing-reverso` no dashboard
2. Colar a transcrição do vídeo que você quer analisar
3. Clicar **Gerar briefing** — API inferem o briefing original via LLM
4. Resultado aparece: estratégia usada, conflito central, tese implícita
5. Usar o briefing gerado como ponto de partida de nova sessão ou repositório de referência

6.12 Preparar reels a partir de vídeo longo (Cortes-prontos)

1. Abrir página `/cortes` no dashboard
2. Colar a transcrição completa do vídeo longo
3. Selecionar as durações desejadas (ex: 30s, 60s, 90s)
4. Clicar **Gerar cortes** — Sônia retorna tabela com cortes prontos para edição
5. Cada corte tem: gancho, desenvolvimento, CTA e minutagem sugerida
6. Copiar ou exportar para o editor de vídeo

6.13 Gate de espelho antes de entregar ao cliente (Gate Espelho)

1. Ter roteiro final aprovado pela equipe interna (pipeline completo rodado)
2. Modo Reunião → convocar Pedro (ou cliente espelho correspondente)
3. Colar roteiro: `@pedro valida esse roteiro antes de eu enviar pro cliente: [roteiro]`
4. Pedro responde com nível de confiança ( alta /  média /  recusa) e observações de voz

5. Ajustar pontos  ou  críticos antes de entregar
6. Sessão fica registrada como evidência de validação pelo espelho do cliente

6.14 Editorial de publicação (Renata)

Cenário A — Via pipeline completo:

1. Montar briefing normalmente e selecionar agentes: Otto → Heitor → Salles → Sônia → Aya
2. No ConfigSidebar, ativar o toggle **editorial (~\$0.20)** sob "Renata"
3. Definir duração (ex: 14 dias) — padrão é 14
4. Clicar **Enviar** — Renata roda automaticamente após Aya
5. Output: calendário editorial de 14 peças com hook, formato, CTA e horário sugerido
6. Descartes salvos em `outputs/renata/estoque/`

Cenário B — Standalone via Reunião:

1. Modo Reunião → `@renata quero um calendário de 21 dias para o Pedro`
2. Se contexto raso: Renata faz 3 perguntas de uma vez (material disponível, cliente/duração, objetivo)
3. Responder com as informações — Renata gera o editorial na próxima mensagem
4. Output ≤ 5000 chars com linha narrativa e publicações listadas

Cenário C — CLI direto:

```
python renata_cli.py \  
  --modo pipeline \  
  --dossie outputs/aya/20260507_143000_dossie_hator.md \  
  --duracao 30 \  
  --cliente pedro_abrahao \  
  --inicio 2026-10-01
```

Output: `outputs/renata/<ts>_humano_pedro_abrahao.md` + JSON técnico + descartes.

Atenção: Renata não faz compliance. Se rodar em modo solo, o output humano inclui aviso "passe pelo Heitor antes de publicar". Em pipeline, Heitor já rodou antes.

7. Roadmap

O `PLANO_ACAO_2026-05-05.md` na raiz do projeto contém o plano completo com 9 épicos e 39 tarefas. Resumo do que está no horizonte:

Concluídos. ~~Família de espelhos de cliente~~ ✓ (v1.1), ~~Pedro como gate de qualidade~~ ✓ (v1.2), ~~Memória institucional e saúde~~ ✓ (v1.3), ~~Workflows avançados~~ ✓ (v1.4), ~~Inteligência operacional / custo-cap~~ ✓ (v1.5), ~~Multimodal e aprovação de cliente~~ ✓ (v1.6), ~~Camada visual~~ ✓ (v1.7).

Todos os épicos do plano de ação 2026-05-05 foram implementados. O sistema está em v1.7 com 37 tarefas concluídas em 7 épicos. Para próximos ciclos, revisar o backlog e criar novo plano de ação baseado em feedback de uso real.

Camada visual (Épico G). Whiteboards que se preenchem em tempo real, sprites com status físico mais expressivo, mesa de reunião com mic destacado quando alguém fala.

Cada épico fechado deve atualizar este manual e gerar nova versão de PDF em `docs/releases/`.

8. Apêndice — custos, limites, configuração

8.1 Faixas de custo previstas

Configurado em `core/config.py` :

Agente	Faixa esperada por execução
Otto	\$0.02–0.08
Heitor (sem busca)	\$0.05–0.10
Heitor (com busca)	\$0.20–0.40
Salles	\$0.10–0.25
Sônia (sem busca)	\$0.15–0.25
Sônia (com busca)	\$0.30–0.50
Aya	\$0.05–0.20
Pedro	\$0.05–0.20

Custo total típico de pipeline completo: \$0.50–1.50.

Threshold de alerta de pipeline caro: \$1.00 (`PIPELINE_AVISO_CUSTO_TOTAL_USD`).

8.2 Limites de tamanho

<code>BRIEFING_MIN_CARACTERES</code>	<code>=</code>	<code>50</code>	<code>BRIEFING_MAX_CARACTERES</code>	<code>=</code>	<code>15000</code>
<code>AYA_OUTPUT_AGENTE_MAX_CHARS</code>	<code>=</code>	<code>15000</code>	<code>AYA_DOSSIE_MAX_CHARS_TOTAL</code>	<code>=</code>	<code>100000</code>
<code>PEDRO_INPUT_MAX_CHARS</code>	<code>=</code>	<code>20000</code>	<code>SONIA_ROTEIRO_MAX_CHARS</code>	<code>=</code>	<code>30000</code>

8.3 Modelo padrão

`claude-sonnet-4-6` configurado em `LEMMON_MODELO_PADRAO` (env var) ou `core/config.py` .

Visão (descrição de imagem upload): `claude-haiku-4-5-20251001` .

8.4 Estrutura de pastas

```
lemmon-agentes/
├── agentes/           # implementações dos 6 agentes
├── core/              # base, custos, validador, espelho genérico
│   ├── espelho.py     # EspelhoCliente – classe genérica de espelho de cliente
│   └── limites_espelho.py # avisos de custo pré/pós execução para espelhos
├── prompts/          # system prompts versionados (v1, v2, v3)
├── inputs/            # briefings, dossiês, transcrições
│   └── clientes/      # material primário por cliente espelho
│       └── pedro/      # dossie.md + transcricoes.md do Dr. Pedro Abrahão
├── outputs/           # outputs de execuções (por agente e por cliente)
├── historico/         # sessões salvas (JSON)
├── dashboard/         # frontend Next.js
├── docs/              # este manual e releases
├── onboard_cliente.py # wizard CLI para novo cliente espelho
└── PLANO_ACAO_*.md    # plano de implementação
```

8.5 Variáveis de ambiente

Var	Default	Função
ANTHROPIC_API_KEY	(obrigatório)	Chave da API
LEMMON_MODELO_PADRAO	claude-sonnet-4-6	Modelo dos agentes
LEMMON_LOG_LEVEL	INFO	Nível de log
NEXT_PUBLIC_API_URL	http://localhost:8000	URL backend (frontend)

8.6 Como subir o sistema localmente

Backend:

```
cd lemmon-agentes
source .venv/bin/activate # ou criar com python -m venv .venv
pip install -r requirements.txt # se existir
uvicorn api_server:app --reload --port 8000
```

Frontend:

```
cd lemmon-agentes/dashboard
npm install
npm run dev # http://localhost:3000
```

9. Como atualizar este manual

9.1 Quando atualizar

- Sempre que uma tarefa do PLANO_ACAO for fechada
- Quando uma função nova for adicionada
- Quando um agente for criado, alterado significativamente, ou removido
- Quando comportamento documentado aqui mudar

9.2 Como atualizar

1. **Editar** `MANUAL_SISTEMA.md` — fazer as mudanças no markdown. Esta é a fonte de verdade.
2. **Atualizar cabeçalho:**
3. ****Versão atual:**** `vX.Y` — incrementar (v1.0 → v1.1 para mudanças menores; v1.0 → v2.0 para reformulações)
4. ****Última atualização:**** `YYYY-MM-DD`
5. **Adicionar entrada no** `## Histórico de versões` (no topo da seção, novidades sempre primeiro): ``` ### v1.1 — 2026-05-15`

O que mudou: - Adicionado agente Marcia (pós-produção) - Modo remix documentado em §6
``

1. **Atualizar** `CHANGELOG.md` com entrada equivalente.
2. **Gerar PDF:** `bash cd /Users/calebe/Documents/lemmon-agentes python docs/gerar_pdf.py`
Saída: `docs/releases/MANUAL_v<versao>_<YYYY-MM-DD>.pdf`

9.3 Convenção de versionamento

Major (v2.0). Reformulação grande do sistema. Mudança de paradigma. Renomeação de agentes principais. Quebra de compatibilidade com sessões antigas.

Minor (v1.1, v1.2). Funções novas. Agentes novos. Modos de operação novos.

Patch (v1.0.1). Correção de doc, ajuste fino de descrição, sem mudança real no sistema.

9.4 Releases nunca somem

Cada PDF gerado em `docs/releases/` permanece para sempre como snapshot histórico. **Não apagar.** Útil para:

- Auditoria do estado do sistema em data X
- Comparação de evolução
- Onboarding de pessoa nova ("o sistema em v1.0 era assim, agora está em v1.5")
- Documentação para cliente externo (Hator pode receber v atual sem ver a próxima ainda em desenvolvimento)

9.5 Atualização contínua é critério de aceite (T92, regra a partir de v1.18)

Regra: toda tarefa que adicionar feature, agente, configuração, modo de operação, página do dashboard, CLI ou comportamento visível ao operador **DEVE atualizar as seções §2 a §8 deste manual** antes do bump de versão. Sem isso, a tarefa não é considerada concluída — mesmo com código funcional, testes verdes e changelog atualizado.

Por quê: o changelog é o diário cronológico do que mudou; o manual é o livro de instruções atualizado. São documentos diferentes com função diferente:

- Quem abre o **changelog** quer saber: "o que entrou na v1.X?"
- Quem abre o **manual** quer saber: "como uso o sistema HOJE?"

Se só o changelog é atualizado, em pouco tempo o manual fossiliza enquanto o sistema cresce — foi exatamente o que aconteceu entre v1.0 e v1.17 (motivou T91).

Onde atualizar de acordo com o tipo de mudança:

Tipo de mudança	Seção do manual a atualizar
Agente novo	§2 (subseção dedicada)
Modo de operação novo	§3
Função do dashboard	§4
CLI novo ou alterado	§5
Workflow novo	§6 (receita)
Item de roadmap entregue	§7 (mover de roadmap pra estável)
Custo, limite ou env var	§8

Como verificar antes do PR: ler o diff completo da tarefa, perguntar "alguém usando o sistema veria isso de alguma forma?". Se sim → seção do manual atualizada. Se não (refatoração interna, dívida técnica) → manual permanece igual, marcar tarefa como `Afeta output: NÃO` no commit.

10. Como adicionar um cliente espelho novo

Siga estas 8 etapas para colocar um novo cliente espelho em produção. Após concluir, o cliente aparece no escritório virtual e pode ser convocado em modo Reunião com `@<alias>`.

10.1 Rodar o wizard de onboarding

```
python onboard_cliente.py
# Ou passando argumentos diretamente:
python onboard_cliente.py --id marina --nome "Marina Costa" --nicho "nutricionista"
```

O wizard cria automaticamente: - `inputs/clientes/<id>/dossie.md` — dossiê de posicionamento (template) - `inputs/clientes/<id>/transcricoes.md` — arquivo para transcrições reais - `prompts/<id>_system_v1.md` — system prompt do espelho (template) - `outputs/<id>/` — pasta de outputs

10.2 Preencher o dossiê

Editar `inputs/clientes/<id>/dossie.md` e preencher todas as seções: - Quem é o cliente (formação, trajetória, o que faz hoje) - Nicho e público-alvo (dor principal) - Posicionamento (diferencial, tom de voz, palavras frequentes, o que nunca diz) - Zonas de recusa do espelho IA - Projetos e contexto atual - Níveis de confiança (🟢/🟡/🔴 por tema)

Regra: quanto mais rico o dossiê, mais fiel o espelho.

10.3 Colar transcrições reais

Editar `inputs/clientes/<id>/transcricoes.md` e colar material de voz real do cliente: - Transcrições de vídeos publicados - Trechos de entrevistas ou podcasts - Posts longos aprovados pelo cliente

Formato sugerido: `## [Data] [Fonte] + transcrição.`

10.4 Ajustar o system prompt

Editar `prompts/<id>_system_v1.md` e substituir todos os `(preencher após dossiê)` com as informações reais do dossiê.

Seções a completar: QUEM VOCÊ É, tom de voz, vocabulário, zonas de recusa completas.

10.5 Instanciar o agente Python

Criar `agentes/<id>.py` ou usar diretamente:

```
from core.espelho import EspelhoCliente
from core.config import ESPELHO_CLIENTES_DIR

marina = EspelhoCliente(
    id="marina",
    nome="Marina Costa",
    material_dir=ESPELHO_CLIENTES_DIR / "marina",
    max_tokens=4096,
)
```

Se o cliente precisar de configuração específica (limites, aviso vermelho), criar subclasse em `agentes/marina.py` seguindo o padrão de `agentes/pedro_abrahao.py`.

10.6 Adicionar ao dashboard (snippet TypeScript)

O wizard imprime um snippet TS. Colar em `dashboard/lib/agents.ts` dentro do array `AGENTS`:

```
{
  id: 'marina',
  name: 'Marina',           // primeiro nome
  title: 'nutricionista',
  rpgClass: 'Cliente',
  color: '#0f766e',
  colorDim: '#0f766e20',
  colorText: '#fff',
  deskPosition: { x: 400, y: 140 }, // ajustar conforme layout
  meetingPosition: { x: 440, y: 260 },
  idleQuote: 'TODO: frase de fundo da Marina',
  reuniaoOnly: true,
},
```

Ajustar `deskPosition` e `meetingPosition` para não colidir com agentes existentes. Definir `idleQuote` com frase característica real do cliente.

10.7 Registrar alias de menção

Editar `api/deps.py` e adicionar no dict `AGENTE_ALIAS`:

```

AGENTE_ALIAS: dict[str, str] = {
    ...
    "marina": "marina",    # @marina → agente marina
}

```

E no dict `_make_agent`, adicionar o mapeamento:

```

"marina": Marina,    # importar de agentes.marina

```

Após isso, `@marina` funciona em modo Reunião.

10.8 Testar e ativar

```

# Teste CLI antes de subir no dashboard
python -c "
from agentes.marina import Marina
m = Marina()
r = m.executar('Como você aborda a relação entre dieta e energia?', modo='consulta')
print(r['output_humano'][:300])
print('Custo:', r['custo_total_usd'])
"

```

Se a resposta soar como o cliente real:  espelho ativo. Se soar genérico: revisar dossiê e transcrições (mais material = espelho mais fiel).

10.9 Como atualizar o dossiê preservando histórico (T85)

Quando o cliente evoluir o posicionamento (novo projeto, novo público, tom mudou), não sobrescreva diretamente — archive a versão antiga primeiro:

```

# 1. Arquivar versão atual antes de editar
cp inputs/clientes/pedro/dossie.md inputs/clientes/pedro/historico/dossie_v1.md
cp inputs/clientes/pedro/transcricoes.md inputs/clientes/pedro/historico/transcricoes_v1.md

# 2. Editar os arquivos atuais com o material novo
# 3. Rodar pedro_cli.py para confirmar que o espelho soa correto

# 4. Verificar qual hash está sendo usado nas execuções recentes
grep "material_hash" historico/pedro_abrahao/*.json | tail -5

```

Estrutura esperada (pasta `historico/` criada manualmente — não é rastreada pelo git):

```
inputs/clientes/pedro/
├─ dossier.md          # versão atual (sempre editável)
├─ transcricoes.md     # versão atual
└─ historico/          # criar: mkdir inputs/clientes/pedro/historico
    ├─ dossier_v1.md    # versão anterior arquivada
    └─ transcricoes_v1.md
```

Como rastrear: o JSON de cada execução contém `"material_hash": "abc123def456"`. Se algo soar estranho numa resposta passada, compare o hash com os arquivos em `historico/` para saber qual dossiê estava ativo.

Compatibilidade: sessões antigas (antes de T85) não têm o campo `material_hash`. Qualquer display deve tratar com `session.material_hash ?? "?"`.

Manual mantido como documento vivo · Lemmon Produções · 2026